

MORAA GemVision

Work Audit Log · Phase 2 Financial Core · generated 2026-10-02

How to use this page: tick the box ☐ at the start of a task once you have checked that it is done and done well. Write the status (done, in progress, blocked, not started) by hand in the empty box at the right of each task, and fill in the summary table below. Nothing on this page is pre-filled with a status. The reasons, decisions (D1, D2...) and errors (E1, E2...) are explained once for the whole phase, after the checklist; tasks refer to them by number.

Domain	Tasks	Done	In progress	Blocked	Not started
1. Wallet ledger	3				
2. Payments and refunds	5				
3. Orders and limits	3				
4. Proof and safety nets	2				
Total	13				

Why this phase

Customers pay real money into a prepaid wallet and every image order takes money out of it. The audit found that payments could be lost or kept after a refund (the Razorpay webhook answered OK even when crediting failed, and refunds or chargebacks never left the wallet), that balances had no ledger to prove them, that a customer could be charged twice for one order or credited twice for one payment under a race, and that the daily spending limit was applied only after the customer had been charged. Without this phase the business could not prove what any customer owes or is owed, and one bad day (a retry storm, a crash at the wrong moment) could silently move money.

WHAT WAS DONE, IN SHORT

Every wallet change now writes exactly one ledger row in the same database transaction, and a check proves the sum of the ledger equals each balance. Payments, refunds and chargebacks go through one guarded path that moves money at most once. Money events that cannot be applied are parked for a person instead of being lost. Orders are declined BEFORE charging when today's generation limit leaves no room.

Checklist

1. Wallet ledger

Every rupee in or out has exactly one reason row.

- ☐
- 1.1 Every wallet change writes one ledger row in the same transaction, and the sum of the ledger equals each customer's balance**
-
- FILES** backend/app/models/wallet_transaction.py, backend/app/services/wallet_service.py, backend/alembic/versions/0009*.py
- HOW TO CHECK** Run backend/scripts/wallet_negative_check.py (read-only). Expect: no customer with a negative balance and no customer whose balance differs from the ledger. The automated tests assert this after every money scenario.

- ☐ **1.2 Opening balances of existing customers are written into the ledger by the migration**

FILES backend/alembic/versions/0009*.py

HOW TO CHECK After the migration, pick any customer in the database: their opening_balance ledger row equals their balance before the migration. Not yet applied to production (waiting for the owner's go-ahead).

- ☐ **1.3 A phone number written two different ways is one customer**

FILES backend/app/utils/phone.py

HOW TO CHECK Recharge from +91 98123 45678 and from 919812345678: the money lands in one wallet.

2. Payments and refunds

Money moves at most once and is never lost.

- ☐ **2.1 A payment is credited once even if Razorpay sends it several times at the same moment**

FILES backend/app/api/routes/payment_routes.py

HOW TO CHECK In Razorpay test mode send the same payment event five times quickly: the wallet rises once.

- ☐ **2.2 Refunds and chargebacks take the money back from the wallet (Option A for amounts over the balance)**

FILES backend/app/api/routes/payment_routes.py, backend/app/services/wallet_service.py

HOW TO CHECK Refund a test payment: balance drops by the refunded amount; a refund bigger than the balance empties the wallet and shows a 'pending review' row, never a negative balance.

- ☐ **2.3 A payment with no usable phone number is parked in pending payments and raised, never lost**

FILES backend/app/services/pending_payment_service.py, backend/app/models/pending_payment.py

HOW TO CHECK Send a test payment without a phone: a pending row appears; crediting it later with the right phone adds the money once.

- ☐ **2.4 Missed in-chat (WhatsApp Pay) and payment-link payments are found and credited by background checks**

FILES backend/app/services/whatsapp_pay_service.py, backend/app/services/razorpay_link_reconcile.py

HOW TO CHECK Stop the webhook, pay a test link, restart: within a few minutes the wallet is credited once.

- ☐ **2.5 The webhook answers an error (so Razorpay retries) when crediting really failed**

FILES backend/app/api/routes/payment_routes.py

HOW TO CHECK Force a database error during a test payment: the response is not 200 OK, and the retry credits it.

3. Orders and limits

A customer is charged only for an order that can run.

- ☐ **3.1 An order is declined before charging when the daily generation limit or kill switch leaves no room**

FILES backend/app/api/routes/meta_webhook.py, backend/app/ai/image_generation_manager.py

HOW TO CHECK Set MAX_GENERATIONS_PER_DAY very low, tap Catalog Pack: reply says nothing was charged and the wallet is unchanged; the photo can be chosen again later.

- ☐ **3.2 A duplicate WhatsApp message is handled once (processed_messages)**

FILES backend/app/services/message_dedupe.py

HOW TO CHECK Replay the same webhook body twice: one reply only.
- ☐ **3.3 A very large recharge request is refused with a clear message**

FILES backend/app/api/routes/meta_webhook.py

HOW TO CHECK Type 'recharge 999999': the bot refuses and names the maximum.

4. Proof and safety nets

- ☐ **4.1 Tests run on SQLite and PostgreSQL and a phase gate re-checks everything with one command**

FILES backend/scripts/phase_gate.py, backend/tests/

HOW TO CHECK Run backend/venv-local/Scripts/python.exe backend/scripts/phase_gate.py --phase "Phase 2" --live-readonly. Every step passes except 'live database up to date', which fails by design until production is upgraded.
- ☐ **4.2 Production database upgraded through migrations 0009-0013 and the write-freeze steps followed**

BLOCKED BY The owner's go-ahead for the production upgrade (docs/PHASE_2_DEPLOY_RUNBOOK.md).

Key decisions and why

D1 Add a wallet ledger table (wallet_transactions) and keep the balance column, with a database check that they agree.

WHY THIS CHOICE

A ledger gives every rupee a reason and a timestamp, and the invariant 'sum of ledger = balance' can be tested on every scenario. The balance column stays so reads remain fast and old code keeps working.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Compute the balance from the ledger on every read** — not chosen because: Slower, and a bigger rewrite of every place that reads the balance.
- **Keep only the balance column and the audit log** — not chosen because: The audit log is optional text; it cannot prove the balance.

Relates to task(s): 1.1, 1.2

D2 Claim a payment or refund FIRST with a unique audit row (one per payment, one per refunded order), then move the money.

WHY THIS CHOICE

A unique database index decides the winner when two requests race, so money moves at most once even if Razorpay retries or two workers run at the same moment.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Check 'was it already processed?' and then credit** — not chosen because: Two simultaneous requests can both pass the check.
- **In-memory lock** — not chosen because: Does not work across restarts or more than one server process.

Relates to task(s): 2.1, 2.3

D3 When a refund or chargeback is larger than the balance left, take what is there and flag the rest for review (owner's Option A).

WHY THIS CHOICE

The owner chose this. A negative wallet would break the rule that balances are never negative; flagging keeps the money trail honest and asks a person to decide.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Allow a negative balance** — not chosen because: Breaks the database rule and hides the problem.
- **Refuse the clawback entirely** — not chosen because: Lets a customer keep money that was returned to them by the card network.

Relates to task(s): 2.1

D4 No refund for a result the customer merely dislikes; refund only when we failed to deliver.

WHY THIS CHOICE

The owner's rule: poor input photos give poor output. Automatic refunds cover failures, not taste.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Refund on request for any dissatisfaction** — not chosen because: Rejected by the owner: cost and abuse risk.

Relates to task(s): 2.2

D5 Decline an order before charging when today's generation limit has no room (peek, then reserve at generation).

WHY THIS CHOICE

Previously the customer was charged and then refunded after the limit blocked the order. Declining first never takes money it will have to give back.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Charge first and refund on block** — not chosen because: A refund is a second money movement that can itself fail or be delayed.

Relates to task(s): 3.1

Errors faced

E1 Refund processed before the credit it reverses (retry within the same payment)

Cause: A refund event could arrive before the payment-captured event had been credited.

Fix: The refund handler answers 'retry later' (503) for up to six hours until the credit exists, then takes the money back exactly once.

Task: 2.1

E2 test_postgres_semantics flaky: expected waiter in pg_stat_activity but none found

Cause: The test polled the database inside one transaction, so it saw a frozen snapshot and never saw the other connection waiting.

Fix: Poll with autocommit so every look sees the current state. A test fix, not a product bug.

Task: 4.1

E3 WhatsApp Pay and Razorpay could both credit the same recharge

Cause: Both channels can report the same rupees for the same customer at about the same time.

Fix: WhatsApp Pay skips its own credit only when the Razorpay path will really credit; the pending row is closed either way.

Task: 2.4